INVENTORY MANAGEMENT SYSTEM

CT70A3000 Software Maintenance

Author: Thet Htar Zin
Date: 26.03.2026

GitHub link: https://github.com/thethtarzin111/inventory-management-system
Demonstration video link: software maintenance project demo-20260326_223404-Meeting Recording.mp4

Table of Contents

1. Introduction 3
2. Code Comprehension 3
3. Refactoring Implementation 6
4. New Feature Implementation 6
5. Testing Implementation 7
6. Design Decisions 8
7. Evaluation 8

1. Introduction

This document describes the work done for the CT70A3000 Software Maintenance course project that took place in Sprint 2026. The course final project involved working on the given Python-based Inventory Management System (IMS). This system is a desktop application which allows businesses to manage their employees, suppliers, products and customer billing through a graphical user interface (GUI), built utilizing Tkinter and a local SQLite database.

The project was divided into four distinct parts/tasks. Task 1 (T1) required analysing the given codebase and producing comprehensive documentation with the support of diagrams and an architectural description. Task 2 (T2) required enhancing code quality through a technique called refactoring. This includes simplifying complex methods, removing duplicated code, and fixing bugs. Task 3 (T3) required extending the system with an additional new feature requested by hypothetical stakeholders. Task 4 (T4) required writing a test suite covering a span of unit, integration and regression tests using pytest.

The original codebase had several serious quality issues including repeated database connections, SQL injection vulnerabilities, unclosed database connections and a logic bug in product validation. During refactoring, these issues were carefully addressed. Additionally, a new feature added was a Low Stock Alert system which brings real-time inventory monitoring directly from the dashboard. All source code is version-controlled and available on GitHub at the link provided on the cover page.

2. Code Comprehension

As illustrated in the diagrams below, the system follows a modular single-tier architecture. All components run locally with no network dependencies. Moreover, the system uses Tkinter for the graphical user interface. As for the embedded database, SQLite is considered and used. Pillow is leveraged for image rendering.

As shown in Figure 1, the entry point is dashboard.py . This starts the main window and launches all other modules as Top level windows on demand through the left-side navigation menu. This application is divided into four layers as shown in Figure 3. The entry layer contains only dashboard.py. The UI layer consists of seven module classes known as employeeClass, supplierClass, categoryClass, productClass, billClass, salesClass and LowStockClass. Each module class manages its own form, table and other business logics independently. Next, the data layer is centralized through db_helper.py. This exposes a single get_db_connection() function while returning a connection to the local SQLite database ims.db. The external libraries layer include tkinter, PIL, sqlite3, os and time.

Similarly, each UI module follows the same structural pattern which is a form for data entry, a Treeview table for displaying existing records, and CRUD operation buttons. As shown in Figure 2, the most complex module is the billClass which coordinates the shopping cart, calculator, bill generation and inventory deduction in a single workflow. The newly added/extended feature as required in T3 is LowStockClass. This provides real-time stock monitoring by querying products with quantity at or below a configurable threshold that's predefined as 5 units.

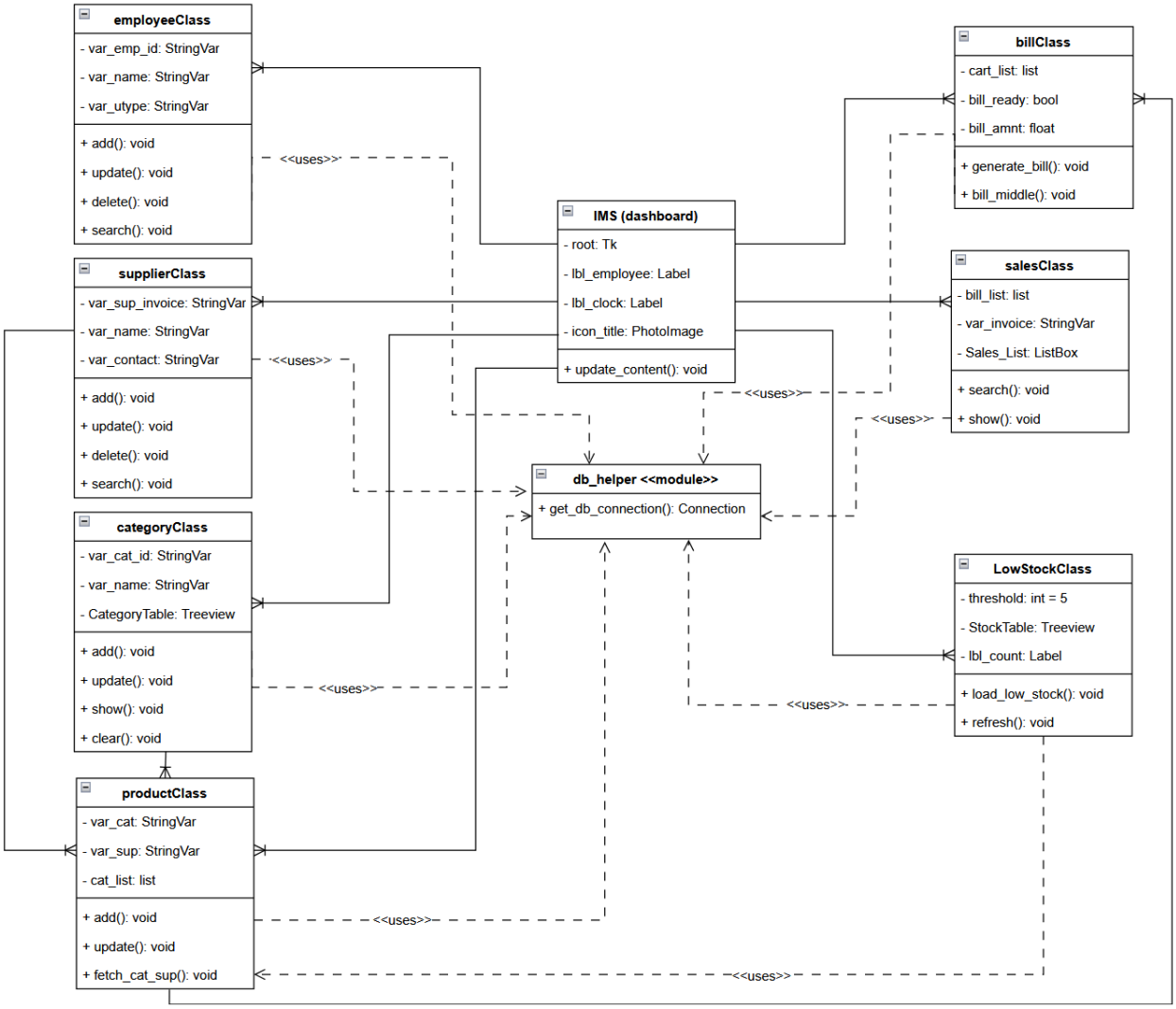

Figure 1: Class diagram

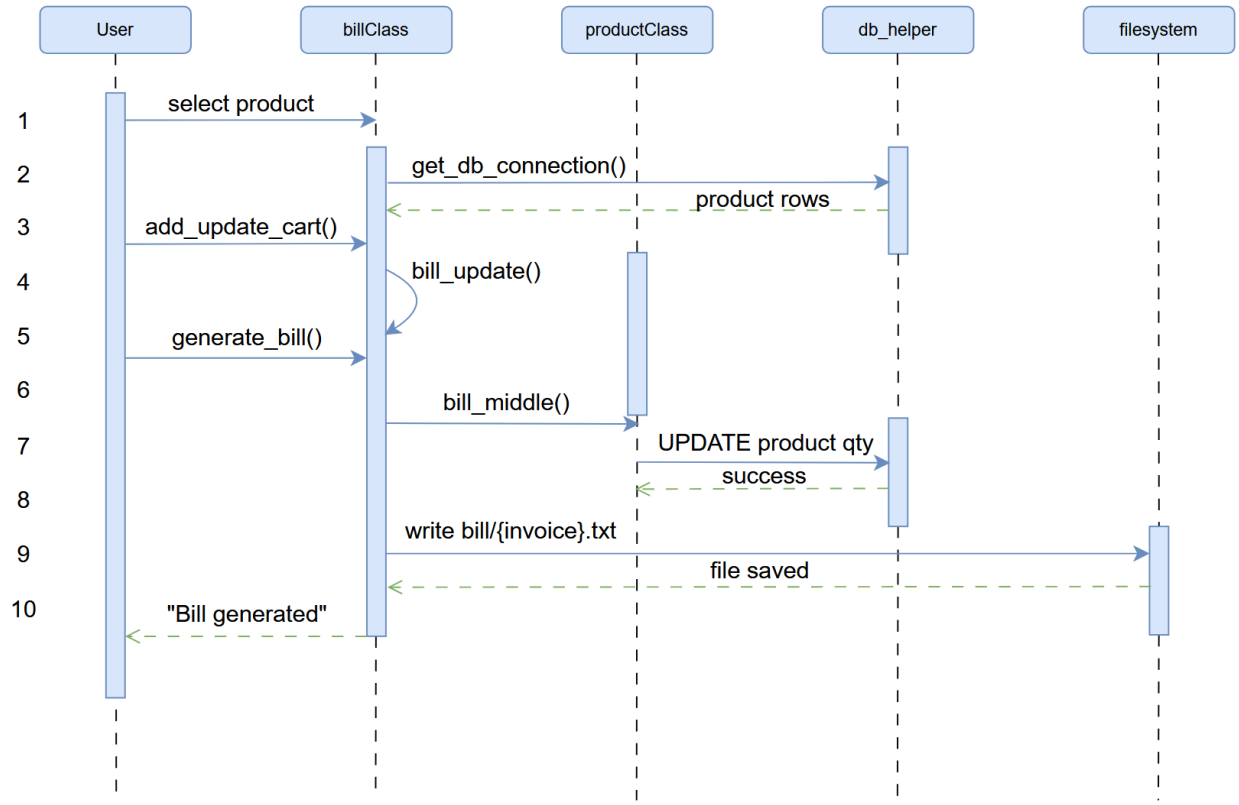

Figure 2: Sequence diagram

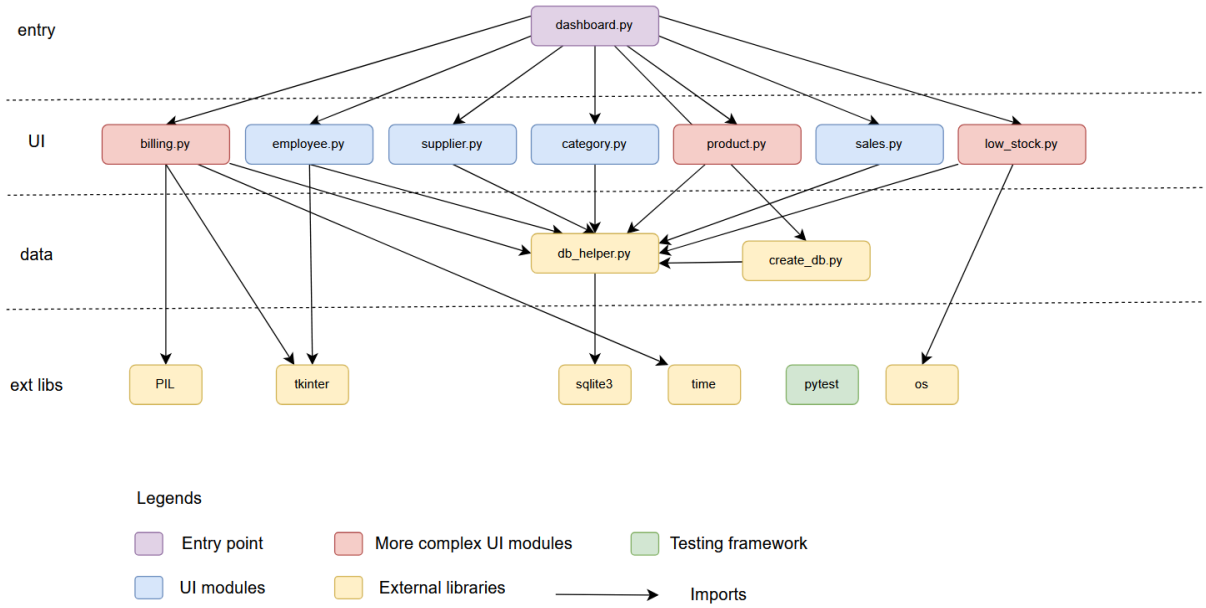

Figure 3: Dependency graph

3. Refactoring Implementation

This section of the report focuses on refactoring the code. The refactoring task emphasized on improving the quality of the code, removing duplications and fixing existing bugs without changing any external behaviour of the system.

The most renowned change was the introduction of db_helper.py. This is a centralized database connection module. In the original codebase, every method in every file calls its own database connector sqlite3.connect(r'ims.db') with a hardcoded path. This pattern was repeated more than 30 times across the entire project. This is not a good programming practice. Therefore, in the refactored code version, all of these duplications are replaced with a single get_db_connection() function so that the database path is resolved dynamically using os.path. This ensures that the application works regardless of the working directory.

Moreover, all database methods were updated with try/finally blocks with con.close() in the finally clause. The original given code never closed any database connection which risked database locking errors especially on the Windows. SQL injection vulnerabilities were also fixed in the search() methods of employee.py and product.py. In the original code, the queries were constructed using string concatenation such as "WHERE name LIKE '%" + user_input + "%'", which is seriously vulnerable to SQL injection. These were replaced with parameterised queries using WHERE name LIKE ? with (f"%{search_txt}%",) as the parameter.

Additionally, a logic bug was found in product.py which compares the StringVar object (self.var_sup == "Select") , resulting in always False, meaning invalid products could be saved. This was fixed by changing it to self.var_sup.get() == "Select". This gets the actual string value and compares it. Also, chk_print was renamed from an integer to a proper boolean bill_ready in billing.py. This extracts the discount rate as a named constant DISCOUNT_RATE = 0.05 and replaces repetitive Treeview column setup with zip() loops in employee.py and product.py. A typo was also fixed (blll_list) to bill_list in sales.py.

4. New Feature Implementation

For task 3 (T3), the new feature implemented is a Low Stock Alert system, extended as a response to a hypothetical stakeholder request for better inventory visibility. The feature alerts users/businesses when product stock levels fall at or below a threshold of 5 units. The feature consists of two components.

The first component is a new dedicated window in a newly created file low_stock.py as LowStockClass. This window shows a color-coded table of all products at or below a threshold. The rows highlighted in red indicate the items that are out of stock. Rows in orange means items are critically low in stock (1-5 units). The window also includes a Refresh button to reload live data and shows a count of low stock items at the bottom.
The second component is an extension or additional integration in the dashboard.py. A new “Low Stock Items” stat card was added to the dashboard. It displays the current amount of low stock products. When

any product number falls at or below the threshold, both the Total Product card and the Low Stock ITems card change to red as a visual warning. When all products are sufficiently in stock, the Low Stock card turns to green. Both cards are clickable and open the Low Stock window directly. A Low Stock button was also added to the left navigation menu.

As for the low stock query, CAST(qty AS INTEGER) <= ? is used with a parameterised threshold value. This correctly handles all quantities that are stored as text in the database. The dashboard's update_content() method refreshes this count every 200 milliseconds along with the existing clock update. Therefore, alerts appear in real time without needing manual refresh.

5. Testing Implementation

To implement the test suite, pytest was used and organized into three separate files within a tests/ directory. This follows standard Python project structure instead of putting all the tests in a single file. A shared conftest.py file defines a db fixture that creates a SQLite database for each test. This promotes complete test isolation meaning no test can affect another. The production database is never touched.

The unit tests in test_unit.py cover a span of eight individual database operations across three test classes. First, TestCategoryUnit tests inserting a new category, detecting duplicates, and deleting by ID. Next, TestProductUnit tests inserting a product, updating quantity after a sale, and verifying that a product's status changes to Inactive when its quantity reaches zero. This mirrors the logic in billing.py's bill_middle() method. Then. TestLowStockUnit tests that the low stock query returns only products at or below the threshold and returns nothing when all products are well stocked.

Two complete workflows are tested by running the integration tests in test_integration.p. The scenario 1 tests the full product lifecycle. This includes adding a category, adding a supplier, adding a product linked to both, simulating a sale, and verifying the final quantity and status. Next, the second scenario tests the low stock alert flow end-to-end. This includes when a product starts above the threshold, a sale drops it below the threshold, and both the dashboard count query and the low stock window query are verified to reflect the change correctly.

Finally, to verify the four bugs fixed during refactoring remain fixed, the regression tests from test_regression.py are run. These cover the duplicate employee ID check, the parameterised search query replacing the original SQL injection vulnerability, the product validation fix using .get() on StringVar objects, and the database connection closure guarantee using try/finally.

6. Design Decisions

Throughout the project, various key design decisions were made. First, when making decisions on database access, centralizing database access through db_helper.py was chosen over other alternatives such as a database class or ORM. Since the project was on a small scale, a lightweight helper function was the most suitable solution because it reduces duplication without introducing unnecessary complications.

When considering the low stock threshold, it was decided to implement it as a module-level constant LOW_STOCK_THRESHOLD = 5 (defined in both dashboard.py and low_stock.py) rather than storing it in the database. The reason is to keep the feature as simple and easy as possible to understand. For future development, this value could be moved to a settings table to provide user configuration without touching any code.

Next, when considering the test suite, it was decided that the suit uses an in-memory SQLite database rather than mocking because :memory: mode from SQLite provides a real database environment. This also accurately reflects production behaviour while being completely isolated as well as fast. On the other hand, mocking would have demanded a more complex setup and could have hidden real SQL errors.

Lastly, the project structure was reorganized to place all application source files in an app/ subdirectory and all tests in a tests/ subdirectory. This separation follows standard Python project conventions and promotes good programming practices.

7. Evaluation

According to T1. Code Comprehension, the codebase was analysed first. Then, it was documented through three diagrams. The first diagram being a class diagram, shows all eight classes with typed attributes, visibility modifiers, and multiplicity relationships. Next, a sequence diagram illustrates the complete bill generation workflow across five participants. Finally, a dependency graph shows the four-layer architecture of the system. The architecture description summarises the system's structure and main components.

As for T2. Refactoring, the refactoring successfully enhanced code quality across all six modified files. Moreover, the introduction of db_helper.py eliminated over 30 repeated database connection calls. SQL injection vulnerabilities were also fixed in two files. Additionally, a logic bug in product validation was corrected. Connection management was improved with try/finally blocks. As a result, the refactored code is more maintainable, more secure, and easier to extend, following standard Python practices.

To satisfy the requirements of T3. New Feature, the Low Stock Alert feature was successfully implemented and integrated into the existing system. The feature required modifications to dashboard.py and the addition of a new low_stock.py module. The implementation correctly queries live data, handles the text-stored quantity field, and provides both passive dashboard alerts and an active dedicated window.

The feature behaves as a real stakeholder requirement would (although in this project, it was a hypothetical one). It is visible at a glance on the dashboard and provides detailed information on demand.

Finally, for T4. Testing, a comprehensive test suite of 14 tests was written covering unit, integration, and regression testing. All tests pass. The test suite validates core database operations, complete user workflows, and confirms that refactoring bugs do not regress. The use of in-memory fixtures ensures tests are fast, isolated, and reliable.

Declaration of AI Usage

An AI tool named Claude was used for grammar checking and language refinement of the written report, as well as for learning and better understanding of certain concepts and techniques during development. All diagrams were produced by the author on draw.io.